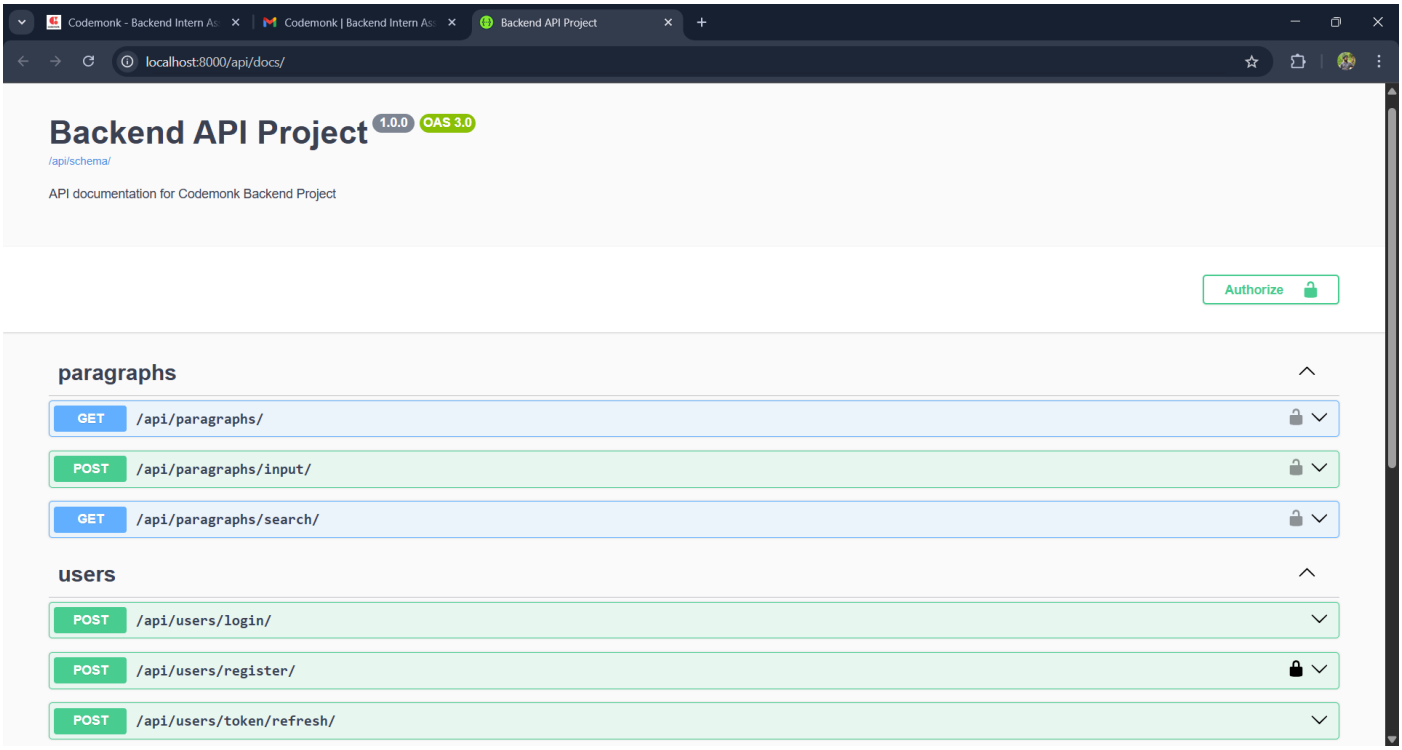
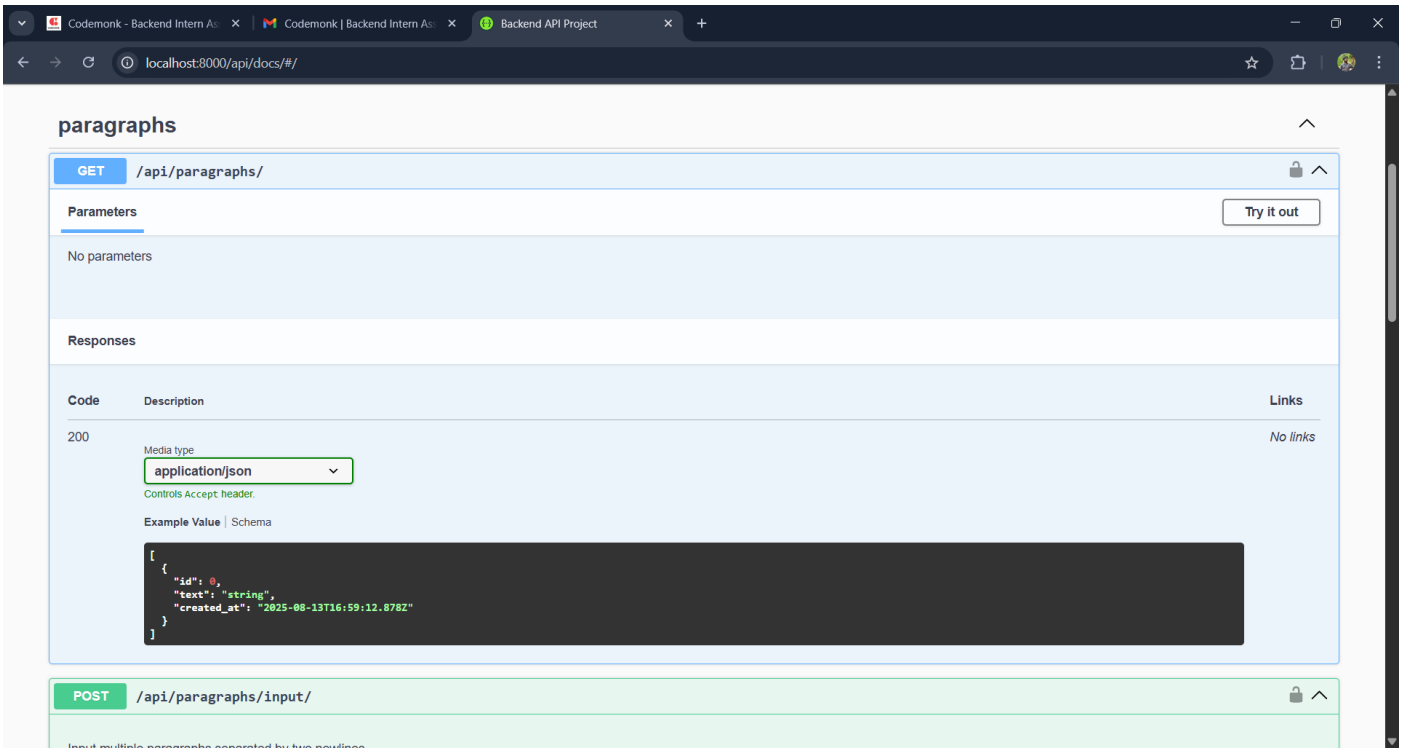
# Django Backend API Project

## 1. API Documentation

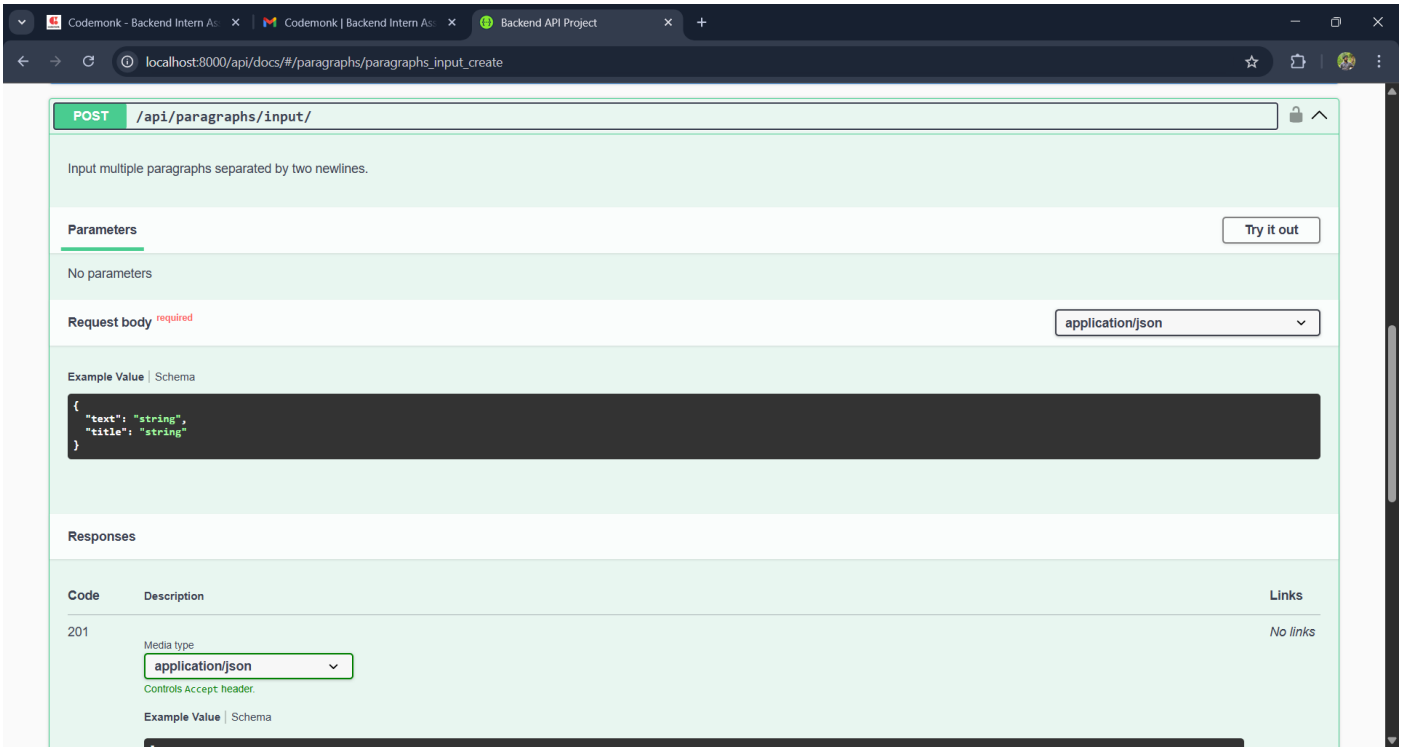



## 2. Paragraph Module

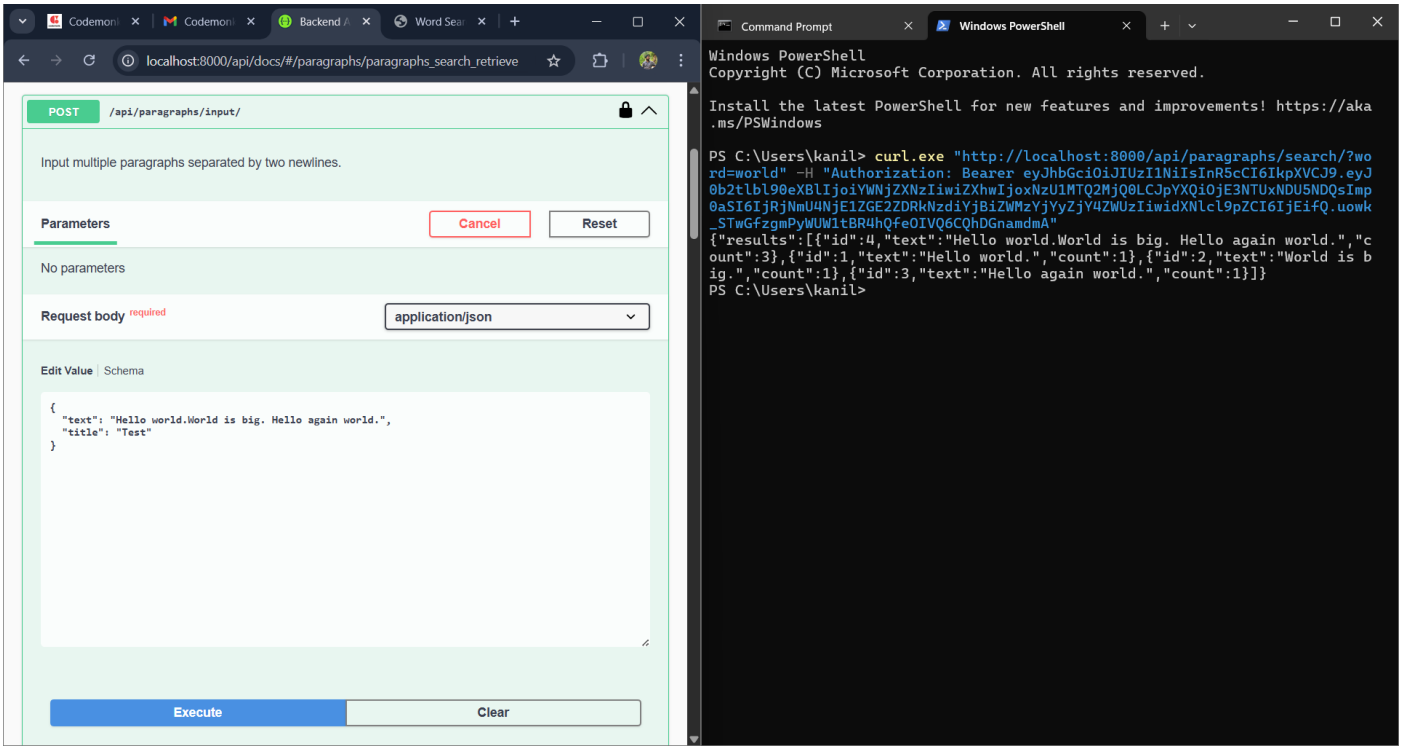
### GET
### /api/paragraphs/



**POST**
**/api/paragraphs/input/**



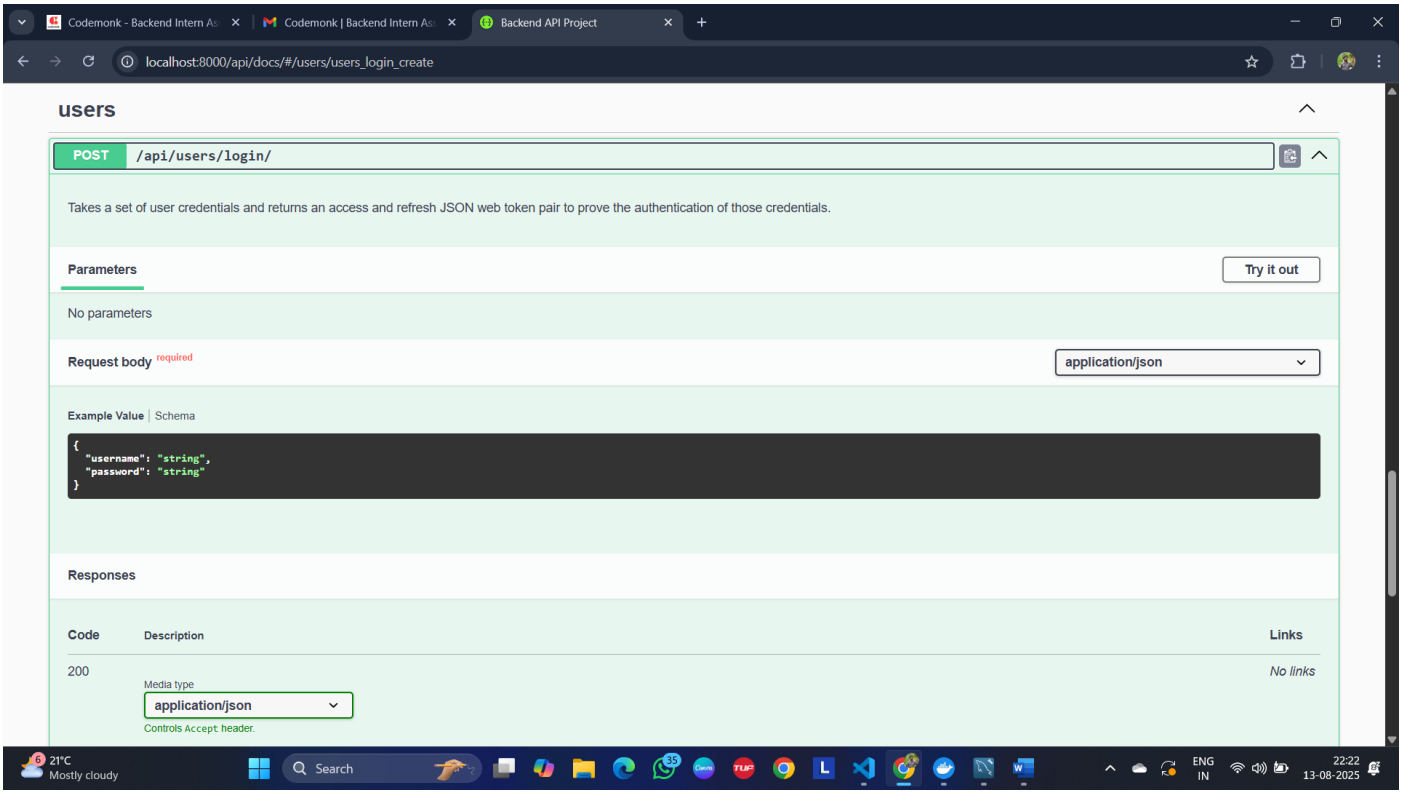

**GET**
**/api/paragraphs/search/**

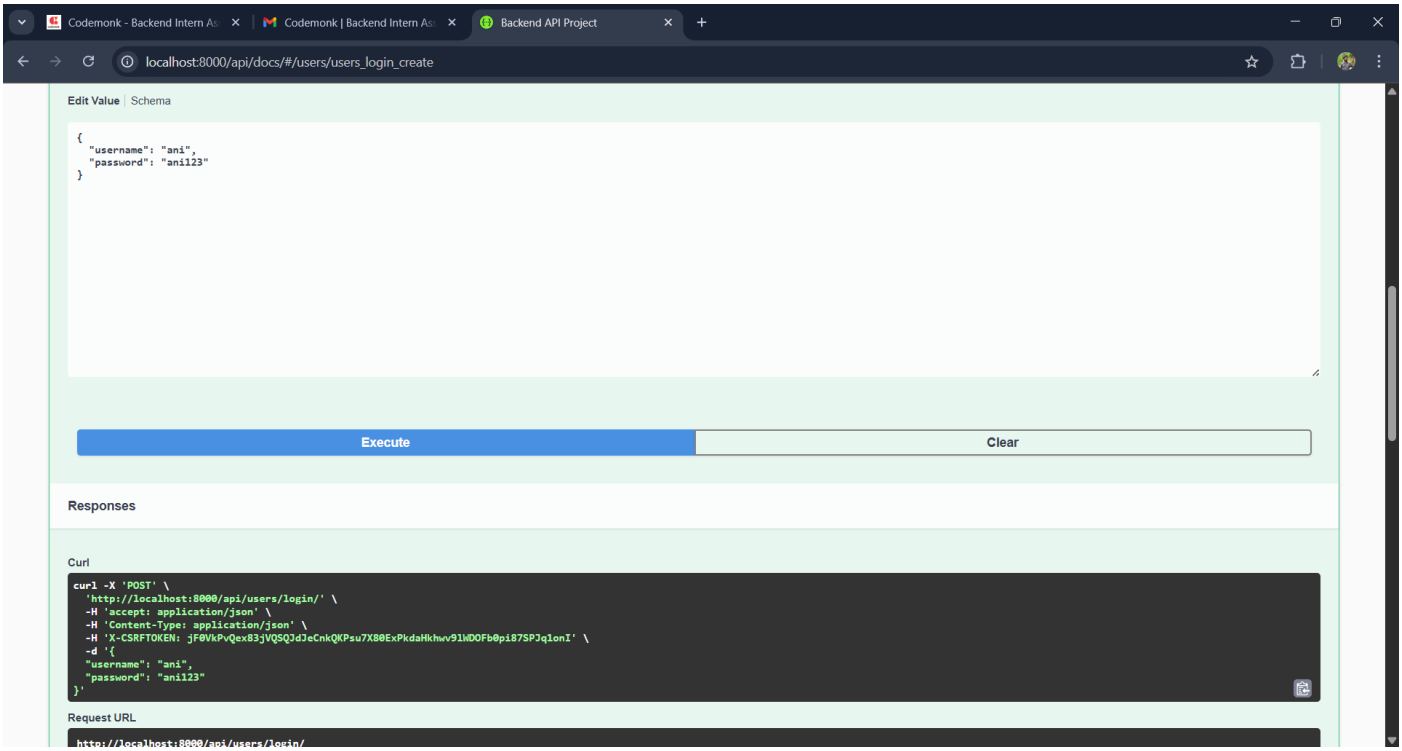


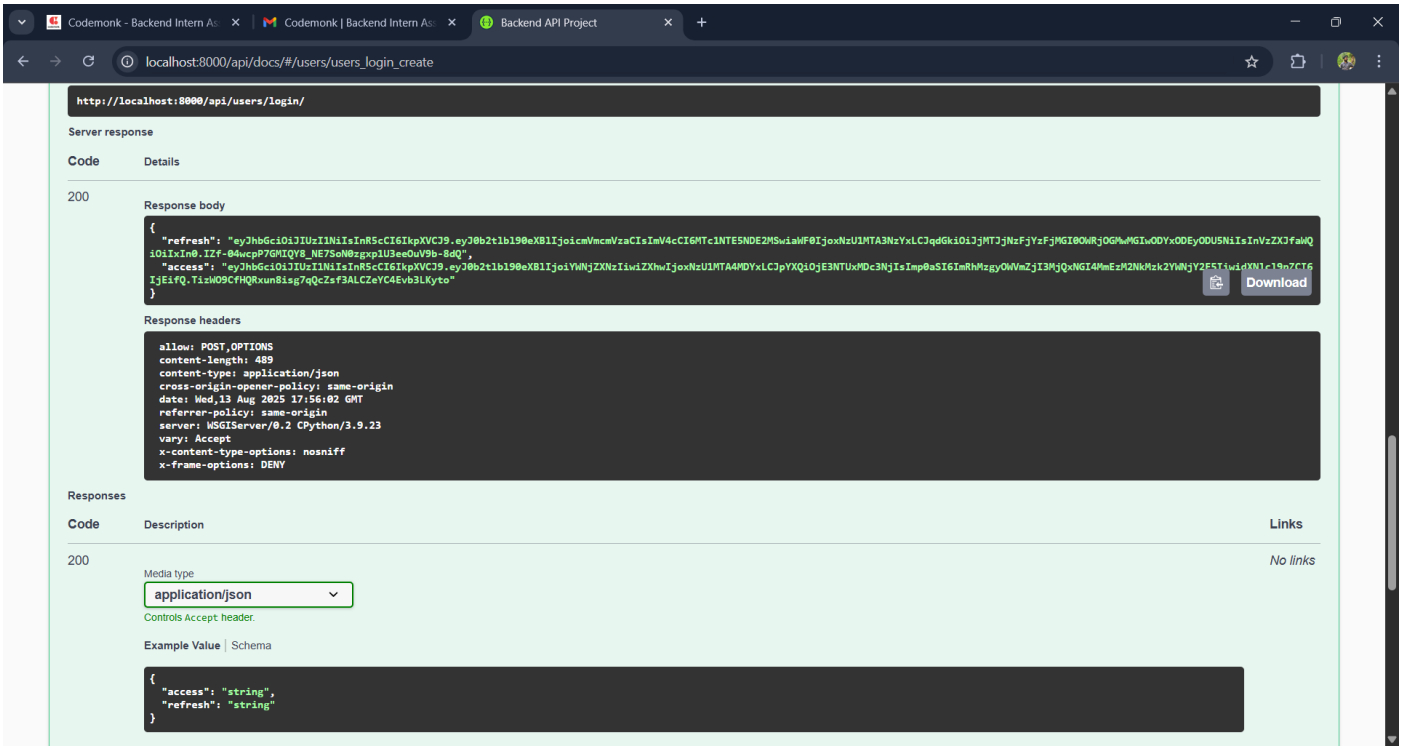
## 3. User Module

I. Login into app

**POST**
**/api/users/login/**

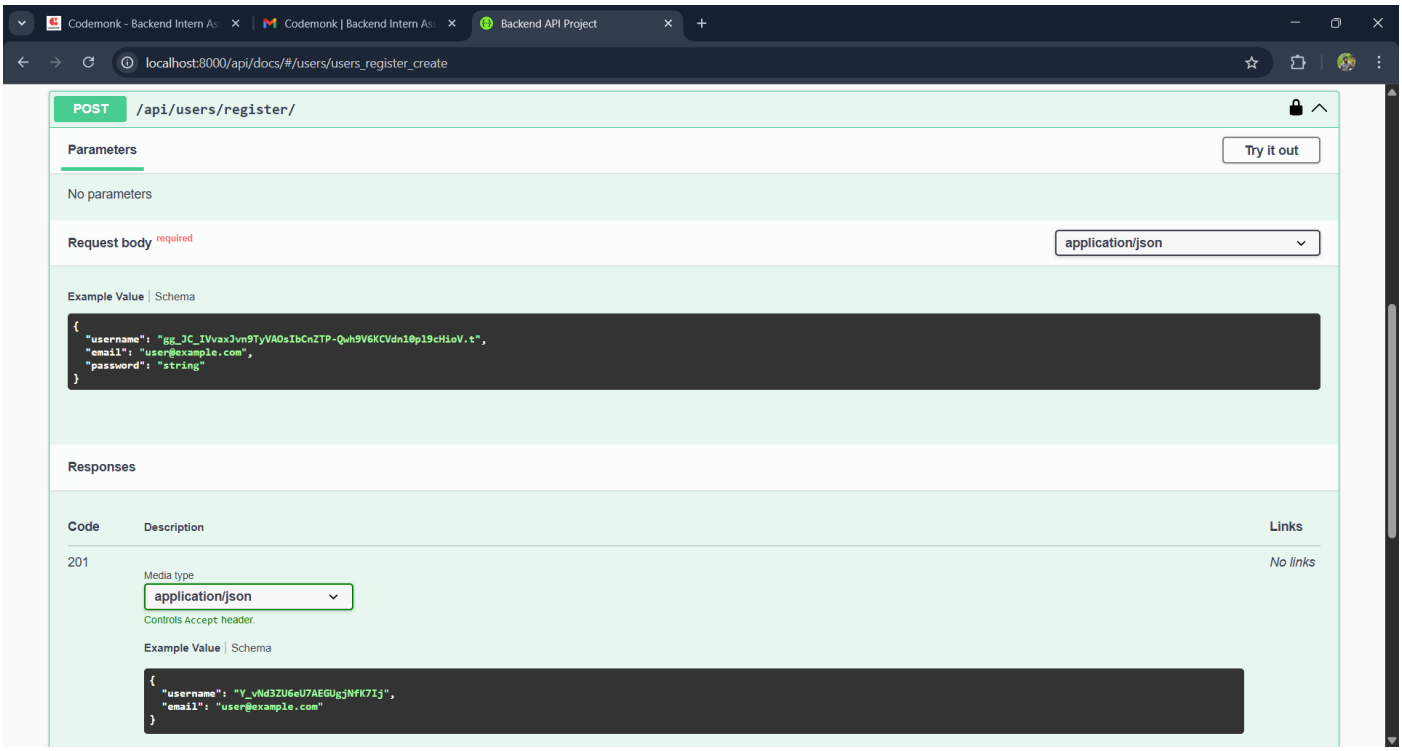


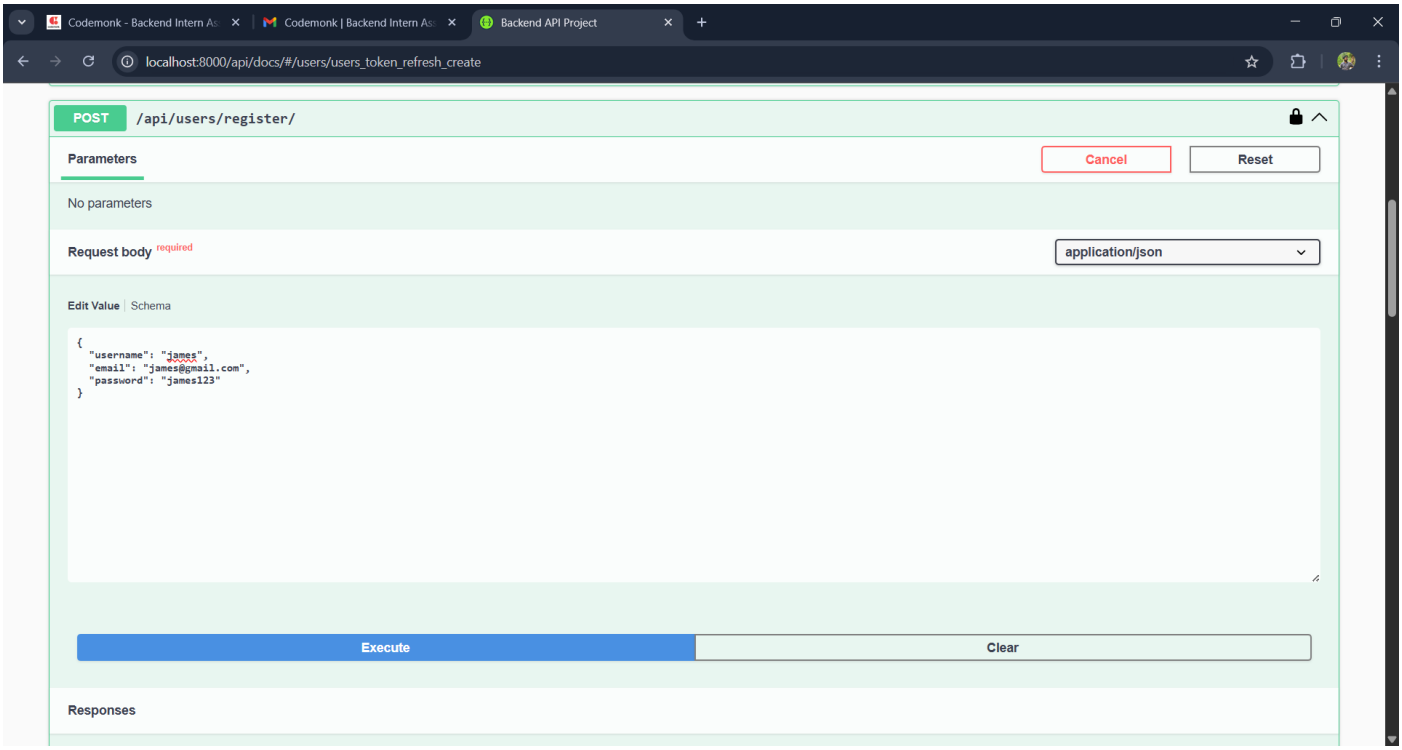



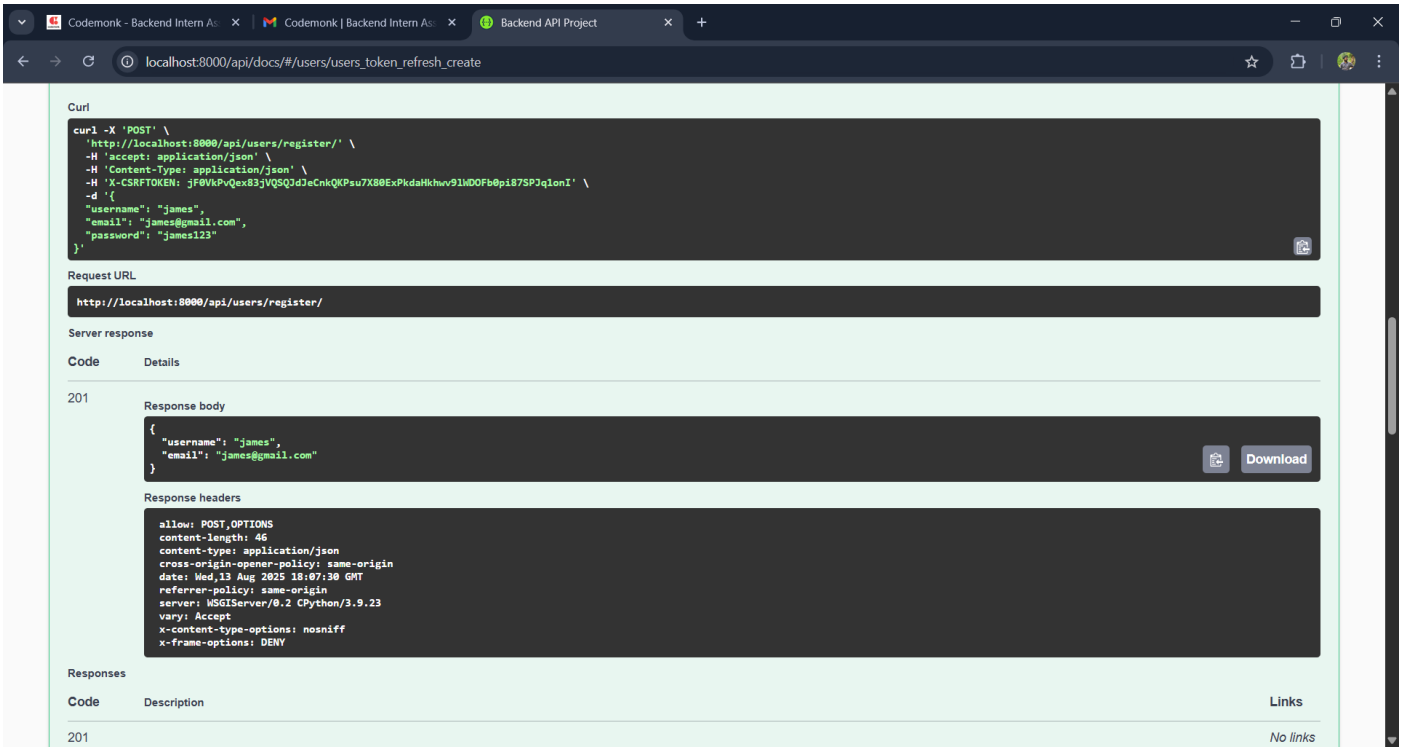



II. Register into app

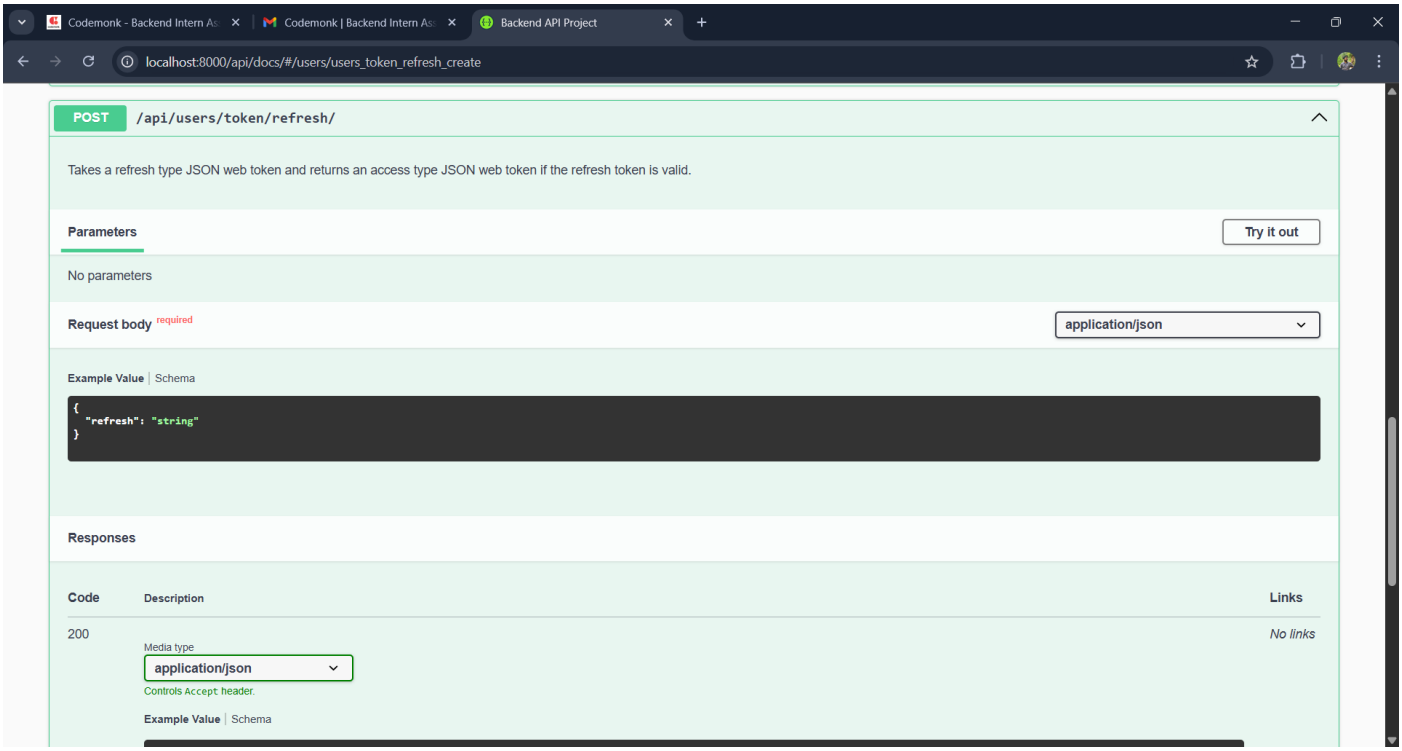
**POST**
**/api/users/register/**




Codemonk - Backend Intern As
Codemonk | Backend Intern As
Backend API Project
localhost:8000/api/docs/#/users/users_token_refresh_create
POST
/api/users/register/
Parameters
Cancel
Reset
No parameters
Request body required
application/json
Edit Value | Schema
{
"username": "james",
"email": "james@gmail.com",
"password": "james123"
}
Execute
Clear
Responses

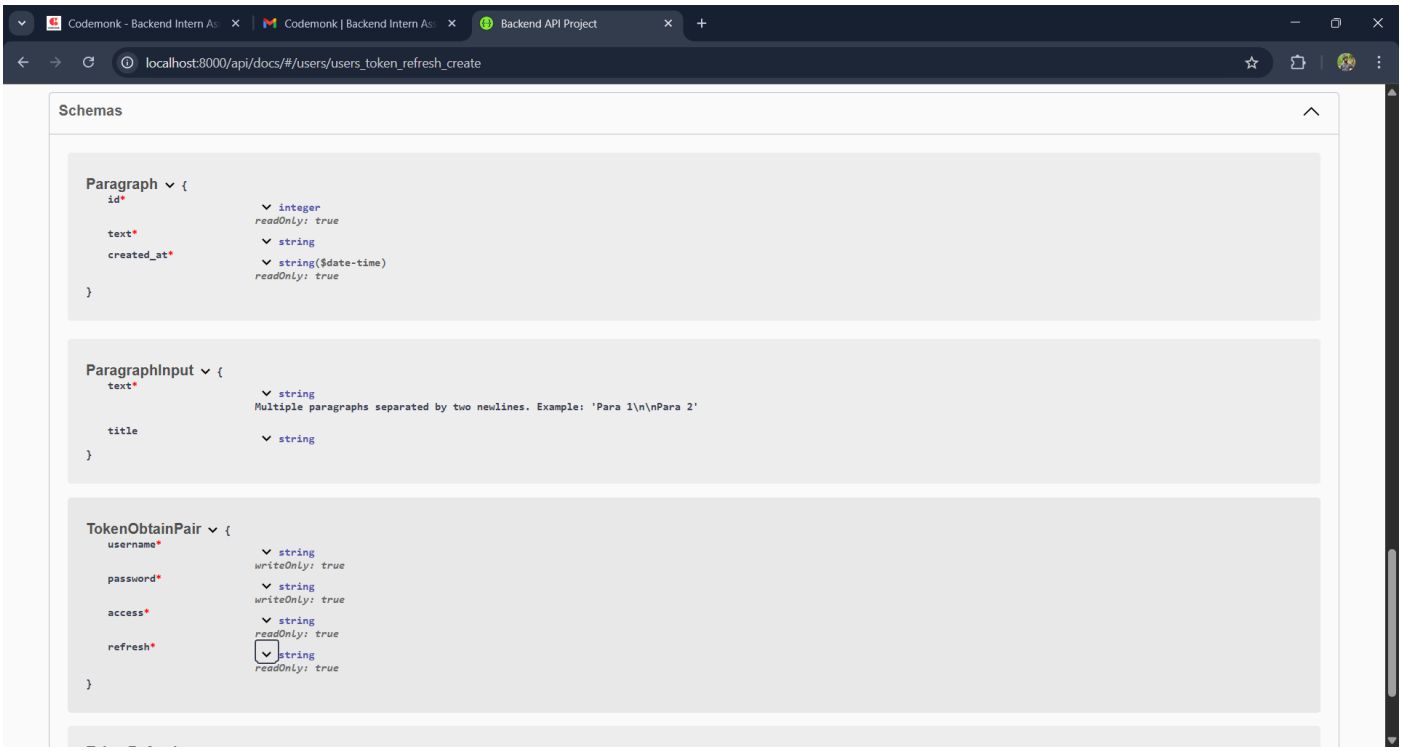

Codemonk - Backend Intern As
Codemonk | Backend Intern As
Backend API Project
localhost:8000/api/docs/#/users/users_token_refresh_create
Curl
curl -X 'POST' \
'http://localhost:8000/api/users/register/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-H 'X-CSRFTOKEN: jF0VkPvQex83jVQSQJdJeCnkQKPsu7X80ExPkdaHkhwv91WDOFb0pi87SPJq1onI' \
-d '{
"username": "james",
"email": "james@gmail.com",
"password": "james123"
}'
Request URL
http://localhost:8000/api/users/register/
Server response
Code
Details
201
Response body
{
"username": "james",
"email": "james@gmail.com"
}
Download
Response headers
allow: POST,OPTIONS
content-length: 46
content-type: application/json
cross-origin-opener-policy: same-origin
date: Wed,13 Aug 2025 18:07:30 GMT
referrer-policy: same-origin
server: WSGIServer/0.2 CPython/3.9.23
vary: Accept
x-content-type-options: nosniff
x-frame-options: DENY
Responses
Code
Description
Links
201
No links

**POST**
**/api/users/token/refresh/**

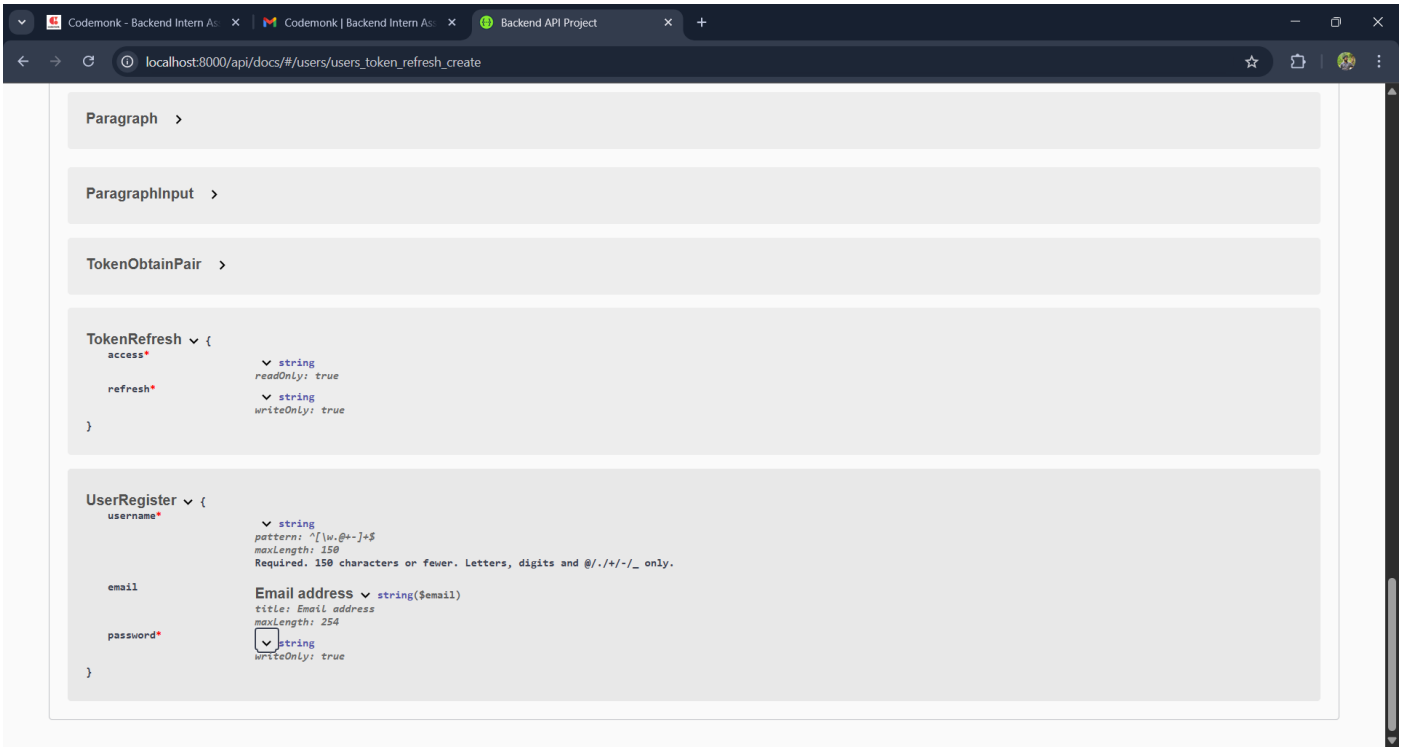



## 4. Schemas

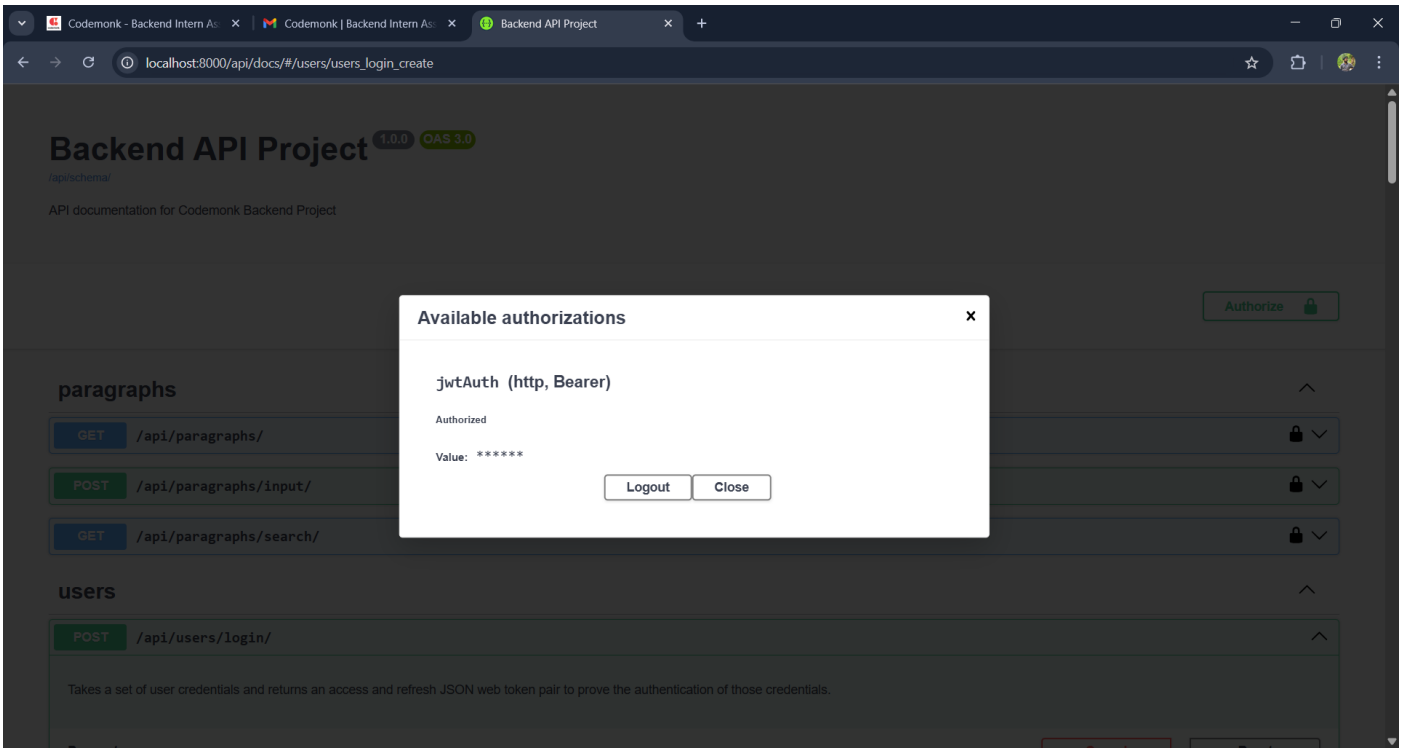
Paragraph, ParagraphInput, TokenObtainPair



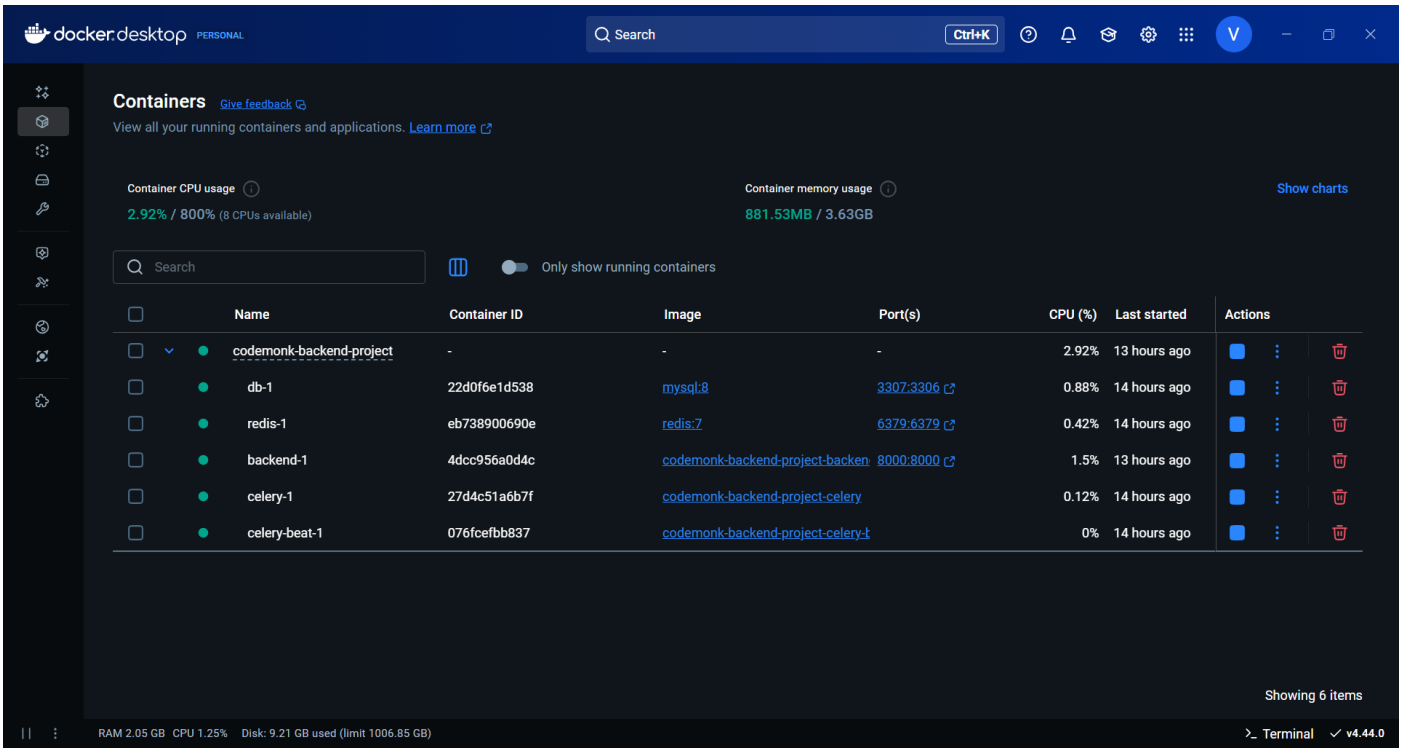
TokenRefresh, UserRegister




## 5. Authorize



## 6. Docker Compose




## 7. Source Code

### Backend Module:

- codemonk-backend-project\backend**settings.py**

```
"""
Django settings for backend project.

Generated by 'django-admin startproject' using Django 4.2.23.

For more information on this file, see
https://docs.djangoproject.com/en/4.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/4.2/ref/settings/
"""

from pathlib import Path
import os  # used for environment variables and docker check

# build paths inside the project (BASE_DIR points to the root folder of the project)
BASE_DIR = Path(__file__).resolve().parent.parent

# development settings

# secret key for django - keep this secret in production
SECRET_KEY = 'django-insecure-u^q^53mr1i4+(1oca4q2-4)8#61t@k!z@jriwwoq3qbtp!29te'

```

```
# debug mode - should be False in production
DEBUG = True

# list of allowed hostnames/domain names for the app
ALLOWED_HOSTS = []

# application definition

# list of installed apps: includes django defaults, third-party apps, and our own apps
INSTALLED_APPS = [
    # django default apps
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # third-party apps
    'rest_framework',            # django rest framework for api building
    'rest_framework_simplejwt',  # jwt authentication for securing apis
    'drf_spectacular',           # api documentation generator (openapi/swagger)

    # custom apps
    'users',                     # handles user registration, login, and profiles
    'paragraphs',                # manages paragraph storage and search features
]

# middleware: processes requests and responses
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',           # security headers and features
    'django.contrib.sessions.middleware.SessionMiddleware',    # session management
    'django.middleware.common.CommonMiddleware',               # common request/response handling
    'django.middleware.csrf.CsrfViewMiddleware',                # csrf protection for forms
    'django.contrib.auth.middleware.AuthenticationMiddleware', # associates users with requests
    'django.contrib.messages.middleware.MessageMiddleware',    # messaging framework
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  # prevents clickjacking attacks
]

# root urls module
ROOT_URLCONF = 'backend.urls'

# template engine configuration
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  # template engine
        'DIRS': [],               # extra template directories (none for now)
        'APP_DIRS': True,         # look for templates in each app's templates folder
```

```
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',      # adds debug info to templates
                'django.template.context_processors.request',    # adds request object to templates
                'django.contrib.auth.context_processors.auth',   # adds user/auth info to templates
                'django.contrib.messages.context_processors.messages', # adds messages to templates
            ],
        },
    },
]

# wsgi entry point - used by servers to run the app
WSGI_APPLICATION = 'backend.wsgi.application'

# database
# check if the app is running inside docker
IN_DOCKER = os.path.exists('/.dockerenv')  # used to switch host/port for docker

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',               # database backend
          'NAME': os.getenv('DB_NAME', 'codemonk'),            # db name (env or default)
          'USER': os.getenv('DB_USER', 'codemonk_user'),       # db username
          'PASSWORD': os.getenv('DB_PASSWORD', 'anil209'),     # db password
          'HOST': 'db' if IN_DOCKER else '127.0.0.1',           # db host (docker vs local)
          'PORT': '3306' if IN_DOCKER else '3307',              # db port (docker vs local)
      }
  }

  # password validation rules
  AUTH_PASSWORD_VALIDATORS = [
      {
          'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',  # no passwords too similar to user info
      },
      {
          'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',           # enforce minimum length
      },
      {
          'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',          # block common passwords
      },
      {
```

```
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',  # block all-numeric passwords
    },
]

# celery configuration - used for background task processing
CELERY_BROKER_URL = os.environ.get('CELERY_BROKER_URL', 'redis://redis:6379/0')  # broker (task queue)
CELERY_RESULT_BACKEND = os.environ.get('CELERY_RESULT_BACKEND', 'redis://redis:6379/0')  # backend (stores results)

# rest framework settings
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',  # use jwt auth
    ),
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',  # auto-generate schema for swagger docs
}

# spectacular (swagger/openapi) documentation settings
SPECTACULAR_SETTINGS = {
    'TITLE': 'Backend API Project',  # api doc title
    'DESCRIPTION': 'API documentation for Codemonk Backend Project',  # api doc description
    'VERSION': '1.0.0',  # version number
    'SERVE_INCLUDE_SCHEMA': False,  # don't include schema in served docs
}

# internationalization
LANGUAGE_CODE = 'en-us'  # language setting
TIME_ZONE = 'UTC'  # time zone setting
USE_I18N = True  # enable django's internationalization system
USE_TZ = True  # enable timezone-aware datetimes

# static files (css, js, images)
STATIC_URL = 'static/'  # url path for serving static files

# default primary key type for models
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

- codemonk-backend-project\backend**urls.py**

```
"""
URL configuration for backend project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/4.2/topics/http/urls/
Examples:
```

```
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path, include
from django.shortcuts import redirect

from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView
from drf_spectacular.utils import extend_schema
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

# home view to redirect root path to /api/docs/
def home(request):
    return redirect('/api/docs/')

urlpatterns = [
    path('', home),  # home view
    path('admin/', admin.site.urls),  # django admin interface
    path('api/users/', include('users.urls')),  # user management routes
    path('api/paragraphs/', include('paragraphs.urls')),  # paragraph features routes
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),  # openapi schema
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-
ui'),  # swagger ui
]

```

- codemonk-backend-project\backend**cerely.py**

```
"""
celery configuration for the django backend project.

- sets default django settings module
- creates celery app named 'backend'
- loads settings with 'celery_' namespace
- auto-discovers tasks from installed apps
"""

import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
app = Celery('backend')
```

```
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

```

**Paragraphs Module:**

- codemonk-backend-project\paragraphs**apps.py**

```
from django.apps import AppConfig  # import base app configuration class

class ParagraphsConfig(AppConfig):  # configuration for paragraphs app
    default_auto_field = 'django.db.models.BigAutoField'  # default primary key type
    name = 'paragraphs'  # app name

```

- codemonk-backend-project\paragraphs**models.py**

```
from django.db import models
from django.contrib.auth.models import User


"""
represents a paragraph submitted by a specific user.
stores the paragraph text, the user who submitted it,
and the date/time it was created.
"""
class Paragraph(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='paragraphs')
    text = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Paragraph {self.id} by {self.user.username}"


"""
tracks how many times a specific word appears in all
paragraphs of a particular user. ensures each (user, word)
pair is unique to avoid duplicate counts.
"""
class WordFrequency(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='word_frequencies')
    word = models.CharField(max_length=100)
    frequency = models.PositiveIntegerField(default=0)

    class Meta:
        unique_together = ('user', 'word')

    def __str__(self):
        return f"{self.word}: {self.frequency} (User: {self.user.username})"
```

- codemonk-backend-project\paragraphs**serializers.py**

```
from rest_framework import serializers
from .models import Paragraph

"""
serializes the paragraph model for api responses.
includes paragraph id, text content, and creation date.
"""
class ParagraphSerializer(serializers.ModelSerializer):
    class Meta:
        model = Paragraph
        fields = ['id', 'text', 'created_at']


"""
handles the input for searching a specific word
across stored paragraphs.
"""
class WordSearchSerializer(serializers.Serializer):
    word = serializers.CharField()


"""
handles input for adding multiple paragraphs at once.
paragraphs should be separated by two newlines, and an
optional title can be provided.
"""
class ParagraphInputSerializer(serializers.Serializer):
    text = serializers.CharField(
        help_text="Multiple paragraphs separated by two newlines. Example: 'Para 1\\n\\nPara 2'"
    )
    title = serializers.CharField(required=False)


# serializer for search results with count
class ParagraphSearchResultSerializer(serializers.ModelSerializer):
    count = serializers.IntegerField()  # number of times the word appears

    class Meta:
        model = Paragraph
        fields = ['id', 'text', 'count']
```

- codemonk-backend-project\paragraphs**urls.py**

```
from django.urls import path
from .views import ParagraphInputView, WordSearchView, ParagraphListView

# url patterns for paragraph-related endpoints
urlpatterns = [
    path('input/', ParagraphInputView.as_view(), name='paragraph_input'),  # endpoint for adding paragraphs
```

```
    path('search/', WordSearchView.as_view(), name='word_search'),  # endpoint for searching word occurrences
    path('', ParagraphListView.as_view(), name='paragraph_list'),  # endpoint for listing all paragraphs
]
```

- codemonk-backend-project\paragraphs**views.py**

```
from rest_framework import generics, permissions
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from paragraphs.serializers import ParagraphInputSerializer, ParagraphSerializer, WordSearchSerializer
from paragraphs.models import Paragraph, WordFrequency
from django.db.models import F
from django.contrib.auth.models import User
from rest_framework.permissions import IsAuthenticated
from django.db.models import Count
from drf_spectacular.utils import extend_schema
from rest_framework.generics import ListAPIView

"""
ParagraphInputView:
- Handles POST requests to input multiple paragraphs for the authenticated user.
- Splits input text into separate paragraphs by two newlines.
- Creates Paragraph objects for each paragraph.
- Updates or creates WordFrequency entries for each word in the paragraphs.
- Returns serialized data of created paragraphs.
"""
class ParagraphInputView(APIView):
    permission_classes = [IsAuthenticated]

    @extend_schema(
        request=ParagraphInputSerializer,
        responses={201: ParagraphSerializer(many=True)},
        description="Input multiple paragraphs separated by two newlines."
    )
    def post(self, request):
        serializer = ParagraphInputSerializer(data=request.data)
        if serializer.is_valid():
            paragraphs_text = serializer.validated_data['text']
            paragraphs = [p.strip() for p in paragraphs_text.split('\n\n') if p.strip()]
            created = []
            for para in paragraphs:
                p = Paragraph.objects.create(user=request.user, text=para)
                created.append(p)
                # token and update the words
                words = para.split()
                for word in words:
```

```
                    freq_obj, _ =
WordFrequency.objects.get_or_create(user=request.user, word=word)
                    freq_obj.frequency = F('frequency') + 1
                    freq_obj.save()
            return Response({'created': ParagraphSerializer(created, many=True).data},
status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

"""
WordSearchView:
- Handles GET requests to search for a specific word in the authenticated user's
paragraphs.
- Counts occurrences of the given word in each paragraph.
- Returns the top 10 paragraphs containing the word, sorted by frequency in descending
order.
"""
import re

class WordSearchView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        word = request.query_params.get('word')
        if not word:
            return Response({'error': 'word parameter is required'}, status=400)
        user_paragraphs = Paragraph.objects.filter(user=request.user)
        para_counts = []
        pattern = re.compile(r'\b{}\b'.format(re.escape(word)), re.IGNORECASE)
        for para in user_paragraphs:
            count = len(pattern.findall(para.text))
            if count > 0:
                para_counts.append({'paragraph': para, 'count': count})
        top = sorted(para_counts, key=lambda x: x['count'], reverse=True)[:10]
        return Response({'results': [
            {'id': x['paragraph'].id, 'text': x['paragraph'].text, 'count': x['count']}
for x in top
        ]})

"""
ParagraphListView:
- Provides a list of all paragraphs created by the authenticated user.
- Uses ParagraphSerializer for serialization.
"""
class ParagraphListView(ListAPIView):
    serializer_class = ParagraphSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Paragraph.objects.filter(user=self.request.user)

```

**Users Module:**

- codemonk-backend-project\users**apps.py**

```
from django.apps import AppConfig # import base app configuration class

class UsersConfig(AppConfig): # configuration for users app
    default_auto_field = 'django.db.models.BigAutoField' # default primary key type
    name = 'users' # app name

```

- codemonk-backend-project\users**serializers.py**

```
from django.contrib.auth.models import User
from rest_framework import serializers

"""
UserRegisterSerializer:
- serializer for user registration.
- accepts username, email, and password fields.
- ensures password is write-only (not returned in responses).
- creates a new User instance using Django's create_user method for proper password hashing.
"""
class UserRegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)

    class Meta:
        model = User
        fields = ('username', 'email', 'password')

    def create(self, validated_data):
        user = User.objects.create_user(
            username=validated_data['username'],
            email=validated_data['email'],
            password=validated_data['password']
        )
        return user

```

- Codemonk-backend-project\users**urls.py**

```
from django.urls import path
from .views import RegisterView
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

# URL patterns for authentication-related endpoints
urlpatterns = [
    path('register/', RegisterView.as_view(), name='register'),  # user registration endpoint
    path('login/', TokenObtainPairView.as_view(), name='login'),  # JWT token obtain endpoint
    path('token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),  # JWT token refresh endpoint
```

```
]
```

- codemonk-backend-project\users**views.py**

```
from rest_framework import generics
from django.contrib.auth.models import User
from .serializers import UserRegisterSerializer

"""
- view for user registration.
- uses Django REST Framework's CreateAPIView to handle POST requests for creating new users.
"""
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()  # retrieves all user records from the database
    serializer_class = UserRegisterSerializer  # serializer used for validating and creating user instances

```

**Docker Files:**

- codemonk-backend-project**docker-compose.yml**

```
version: '3.8'

services:
  # mysql 8 database service
  # stores persistent project data (paragraphs, users, word frequencies, etc.)
  # port 3307 is used on the host to avoid conflicts with any local mysql installation
  db:
    image: mysql:8
    restart: always
    environment:
      MYSQL_DATABASE: codemonk        # database name
      MYSQL_USER: codemonk_user       # application database user
      MYSQL_PASSWORD: anil209         # password for mysql_user
      MYSQL_ROOT_PASSWORD: anil209    # root user password
    ports:
      - '3307:3306'                   # host port : container port mapping
    volumes:
      - db_data:/var/lib/mysql        # persistent storage for mysql data

  # redis service
  # used as a message broker and result backend for celery task queue processing
  redis:
    image: redis:7
    restart: always
    ports:
      - '6379:6379'                   # default redis port

  # django backend service
  # runs the web application with database migrations applied on startup
```

```
  backend:
    build: .
    command: sh -c "python manage.py migrate && python manage.py runserver
0.0.0.0:8000"
    volumes:
      - .:/code                       # mount local code directory into container
    ports:
      - '8000:8000'                    # expose django development server
    depends_on:
      - db
      - redis
    environment:
      - DJANGO_SETTINGS_MODULE=backend.settings
      - CELERY_BROKER_URL=redis://redis:6379/0
      - CELERY_RESULT_BACKEND=redis://redis:6379/0
      - MYSQL_DATABASE=codemonk
      - MYSQL_USER=codemonk_user
      - MYSQL_PASSWORD=anil209
      - MYSQL_ROOT_PASSWORD=anil209

  # celery worker service
  # processes background tasks asynchronously using redis as the broker
  celery:
    build: .
    command: celery -A backend worker -l info
    volumes:
      - .:/code
    depends_on:
      - backend
      - redis
    environment:
      - DJANGO_SETTINGS_MODULE=backend.settings
      - CELERY_BROKER_URL=redis://redis:6379/0
      - CELERY_RESULT_BACKEND=redis://redis:6379/0

  # celery beat scheduler service
  # manages and schedules periodic tasks for the project
  celery-beat:
    build: .
    command: celery -A backend beat -l info
    volumes:
      - .:/code
    depends_on:
      - backend
      - redis
    environment:
      - DJANGO_SETTINGS_MODULE=backend.settings
      - CELERY_BROKER_URL=redis://redis:6379/0
      - CELERY_RESULT_BACKEND=redis://redis:6379/0

# volume for persistent database storage
volumes:
  db_data:

```

- codemonk-backend-project**Dockerfile**

```
# Dockerfile
FROM python:3.9-slim

ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1

WORKDIR /code

# installing system dependencies for mysqlclient
RUN apt-get update \
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /code/
RUN pip install --upgrade pip && pip install -r requirements.txt

COPY . /code/

```

**Requirements:**

- codemonk-backend-project\ **requirements.txt**

```
Django>=4.2
mysqlclient
celery
redis
djangorestframework
djangorestframework-simplejwt
drf-spectacular

```

**README:**

- codemonk-backend-project\ **README.md**

````
# Project Setup Instructions

## Prerequisites
- Docker & Docker Compose
- Python 3.9+
- MySQL 8+
- Redis 7+

## Local Development

1. **Clone the repository**
2. **Create and activate a virtual environment:**
   ```powershell
   python -m venv venv
   .\venv\Scripts\activate
   ```
3. **Install dependencies:**
````

```powershell
pip install -r requirements.txt
```
4. **Configure environment variables** (optional, see `docker-compose.yml` for defaults)
5. **Run migrations:**
```powershell
python manage.py migrate
```
6. **Run the development server:**
```powershell
python manage.py runserver
```
7. **Start Celery worker (in a new terminal):**
```powershell
celery -A backend worker -l info
```
8. **Start Celery beat (in a new terminal):**
```powershell
celery -A backend beat -l info
```

## Dockerized Development

1. **Build and start all services:**
```powershell
docker-compose up --build
```

## API Documentation
- Swagger UI: [http://localhost:8000/api/docs/](http://localhost:8000/api/docs/)
- OpenAPI schema: [http://localhost:8000/api/schema/](http://localhost:8000/api/schema/)

## User APIs
- `POST /api/users/register/` — Register a new user
- `POST /api/users/login/` — Obtain JWT token

## Paragraph APIs
- `POST /api/paragraphs/input/` — Input multiple paragraphs (separated by two newlines)
- `GET /api/paragraphs/search/?word=example` — Get top 10 paragraphs with max occurrences of a word

## Code Documentation
- All modules and classes are documented inline.

---

For any issues, please refer to the code comments and API docs.